

Day 9 Cheatsheet

Functions

Why create your own functions? - Cut down on repetitive code (easier to fix things!) - Organize code into manageable chunks - Avoid running code unintentionally - Use names that make sense to you

Syntax for a function is:

```
functionName <- function(inputs) {  
  # Document your inputs here  
  <function body>  
  return(value)  
}
```

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>function()</code>	<code>div_100 <- function(x) x / 100</code>	Create a new function - in this case x is an input and divided by 100.
Base R	<code>lapply()</code>	<code>lapply(some_list, a_function)</code>	Apply a function over a list or vector.
Base R	<code>sapply()</code>	<code>sapply(some_list, a_function)</code>	Apply a function over a list or vector but by default returns a matrix or vector.
dplyr	<code>across()</code>	<code>iris > group_by(Species) > summarise(across(starts_with("Sepal"), ~ mean(.x, na.rm = TRUE)))</code> or <code>iris > mutate(across(contains("Sepal"), round))</code>	Apply a function across groups or columns in a dataframe.
purrr	<code>mutate_if()</code>	<code>iris > mutate_if(is.numeric, round)</code>	Apply a function to specific columns in a dataframe.

* This format was adapted from the cheatsheet format from AlexsLemonade.